

```

import re
import bisect

from pathlib import Path

class LatexIndexParser:
    """Scans raw LaTeX text streams to extract structural index macro parameters cleanly."""

    INDEX_PATTERN = re.compile(r'\\index\{')
    MACRO_START_PATTERN = re.compile(r'\\(newcommand|renewcommand|def|providecommand|DeclareRobustCommand)\b')

    @classmethod
    def parse_file(cls, file_path: str, start_id: int = 1) -> tuple[list[tuple[list, dict]], int]:
        extracted_payloads = []
        path_obj = Path(file_path).resolve()
        norm_path_str = path_obj.as_posix()

        if not path_obj.exists() or not path_obj.is_file():
            return extracted_payloads, start_id

        try:
            full_content = path_obj.read_text(encoding='utf-8')
        except Exception as e:
            print(f"ERROR: Failed to read file {norm_path_str} for indexing: {e}")
            return extracted_payloads, start_id

        # 1. CRITICAL: Standardize line breaks across ALL operating systems (\r\n -> \n)
        # This guarantees that the string layout inside Python perfectly matches QPlainTextEdit
        full_content = full_content.replace('\r\n', '\n').replace('\r', '\n')

        # 2. ANCHOR: Build the immutable baseline offset map immediately from raw, unmutated text
        line_offsets = cls._build_line_offset_map(full_content)
        cls._offset_map = line_offsets # Keep for backward compatibility

        # Scrubbing uses strict 1:1 character index masking (preserving exact \n positions)
        working_content = cls._scrub_macro_definitions(full_content)
        running_id = start_id

        for match in cls.INDEX_PATTERN.finditer(working_content):
            # 3. Capture the absolute starting character index of the macro match
            absolute_index = match.start()

            start_pos = match.end()
            inner_content = cls._extract_balanced_braces(working_content, start_pos)

            if not inner_content:
                continue

            # Split structural terms from formatting flags before level parsing
            clean_inner_content, encap_format = cls._strip_global_encap_safe(inner_content)

            # Process individual layout tiers cleanly
            levels = cls._split_levels_safe(clean_inner_content)

            parts_list = []
            for level in levels:
                level_cleaned = level.strip().replace('\n', ' ')
                level_cleaned = " ".join(level_cleaned.split())

                if not level_cleaned:
                    continue

                display_text = cls._extract_display_string_safe(level_cleaned)
                parts_list.append(display_text.strip())

            if not parts_list:
                continue

        # 4. COMPUTE PRISTINE COORDINATES
        # Uses optimized binary search to map absolute character index to 1-based (line, column)
        line_idx, col_idx = cls._compute_coordinates(absolute_index, line_offsets)

        # Ensure encap defaults to a string 'standard' rather than a database breaking None type
        safe_encap = encap_format if encap_format else "standard"

        # Generate a flawless, concrete unique identifier string
        stable_uid = f"{norm_path_str}:{line_idx}:{col_idx}"

        # 5. UNIFIED DATABASE PAYLOAD SCHEMA
        # Explicitly maps BOTH potential keys ('id', 'unique_id_number') so that your
        # synchronization tables populate beautifully without missing fields.
        uid_dict = {
            "id": running_id,
            "unique_id_number": running_id, # structural alignment with UI slots
            "uid": stable_uid,             # unique string tracking key
            "file_path": norm_path_str,

```

```

        "line_number": line_idx,          # 1-based line integer
        "column_offset": col_idx,        # 1-based column integer
        "absolute_index": absolute_index, # absolute structural position anchor
        "encap": safe_encap              # non-None string value
    }

    extracted_payloads.append((parts_list, uid_dict))
    running_id += 1

    return extracted_payloads, running_id

@classmethod
def _strip_global_encap_safe(cls, text: str) -> tuple[str, str]:
    text = text.strip().replace(r'\string', '')
    brace_level = 0
    for i in range(len(text) - 1, -1, -1):
        char = text[i]
        if char == '}':
            brace_level += 1
        elif char == '{':
            brace_level -= 1
        elif char == '|' and brace_level == 0:
            clean_path = text[:i].strip()
            encap_format = text[i+1:].strip()
            if encap_format in ('(', ')'):
                encap_format = "range"
            return clean_path, encap_format
    return text, "standard"

@classmethod
def _scrub_macro_definitions(cls, text: str) -> str:
    working_chars = list(text)
    text_len = len(text)
    for match in cls.MACRO_START_PATTERN.finditer(text):
        idx = match.end()
        while idx < text_len:
            if text[idx].isspace():
                idx += 1
            elif text[idx] == '[':
                while idx < text_len and text[idx] != ']':
                    idx += 1
                idx += 1
            elif text[idx] == '{':
                end_name_brace = cls._find_closing_brace_index(text, idx)
                if end_name_brace != -1:
                    next_idx = end_name_brace + 1
                    while next_idx < text_len and (text[next_idx].isspace() or text[next_idx] == '['):
                        if text[next_idx].isspace():
                            next_idx += 1
                        elif text[next_idx] == '[':
                            while next_idx < text_len and text[next_idx] != ']':
                                next_idx += 1
                            next_idx += 1
                        elif text[next_idx] == '{':
                            while next_idx < text_len and text[next_idx] != '}':
                                next_idx += 1
                            next_idx += 1
                    if next_idx < text_len and text[next_idx] == '{':
                        idx = next_idx
                        break
                else:
                    break
            else:
                idx += 1

        if idx < text_len and text[idx] == '{':
            end_pos = cls._find_closing_brace_index(text, idx)
            if end_pos != -1:
                # FIX: Preserve both \n and \r characters exactly during string cleaning
                # to keep a 1:1 identical character index map profile.
                for fill_idx in range(match.start(), end_pos + 1):
                    if working_chars[fill_idx] not in ('\n', '\r'):
                        working_chars[fill_idx] = ' '
    return "".join(working_chars)

@classmethod
def _find_closing_brace_index(cls, text: str, start_brace_pos: int) -> int:
    brace_count = 0
    for i in range(start_brace_pos, len(text)):
        if text[i] == '{':
            brace_count += 1
        elif text[i] == '}':
            brace_count -= 1
        if brace_count == 0:
            return i
    return -1

@classmethod
def _build_line_offset_map(cls, raw_text: str) -> list[int]:
    offsets = [0]
    for i, char in enumerate(raw_text):
        if char == '\n':

```

```

        offsets.append(i + 1)
    return offsets

@classmethod
def _compute_coordinates(cls, absolute_pos: int, line_offsets: list[int]) -> tuple[int, int]:
    """Maps an absolute character index to 1-based (line, column) coordinate pairing."""
    # Using binary search optimization via bisect to ensure flawless calculations
    line_idx = bisect.bisect_right(line_offsets, absolute_pos) - 1
    line_num = line_idx + 1
    col_num = (absolute_pos - line_offsets[line_idx]) + 1
    return line_num, col_num

@classmethod
def _extract_balanced_braces(cls, text: str, start_pos: int) -> str:
    brace_count = 1
    current_pos = start_pos
    result_chars = []
    while current_pos < len(text):
        char = text[current_pos]
        if char == '{':
            brace_count += 1
        elif char == '}':
            brace_count -= 1
            if brace_count == 0:
                return "".join(result_chars)
            result_chars.append(char)
        current_pos += 1
    return ""

@classmethod
def _split_levels_safe(cls, text: str) -> list[str]:
    levels = []
    current_part = []
    brace_level = 0
    for char in text:
        if char == '{':
            brace_level += 1
        elif char == '}':
            brace_level -= 1
        if char == '!' and brace_level == 0:
            levels.append("".join(current_part))
            current_part = []
        else:
            current_part.append(char)
    levels.append("".join(current_part))
    return levels

@classmethod
def _extract_display_string_safe(cls, level_text: str) -> str:
    brace_level = 0
    for i, char in enumerate(level_text):
        if char == '{':
            brace_level += 1
        elif char == '}':
            brace_level -= 1
        elif char == '@' and brace_level == 0:
            return level_text[i+1:].strip()
    return level_text.strip()

```